

P11 Caffeza setup and menu import

Model: Sonnet, high effort.

Branch: none. Commit directly to `main`.

Depends on: P10.

1. What to build, in one sentence

Build a reusable setup script that configures a restaurant from one JSON file and a menu import from a CSV file, both running through the real API with a dry run by default, and write Caffeza's own setup file from what we know about them.

2. Module

Phase 2 onboarding, from `docs/BUILD-PLAN.md` section 7. It adds scripts and data files only. No endpoint, model or screen changes.

3. Why

Setting up Caffeza by hand means 34 tables, 2 stations, 23 or more categories, around 150 menu items, 8 payment methods, office accounts and a dozen staff logins, typed

into screens one at a time.

It has to be done on staging for training and again on production on cutover day, identically.

A script that reads one file does it the same way both times, and can be re-run safely after the menu changes.

4. Step 0. Save this prompt and check the repo

1. Save this entire prompt, exactly as given, to `docs/prompts/P11-cafeza-setup-menu-import.md`.
2. `git pull` . `git status` should show only that file.
3. `docs/prompts/README.md` must show P10 as Done. If not, stop.
4. Run `npm test` and record the count. It should match P10's "after" count. If anything fails, stop and tell me.

5. Files to read first

1. `docs/CAFFEZA-PROFILE.md` , all of it. Every value in the setup file comes from here.
2. `docs/DEPLOYMENT.md` section 7, first production setup.
3. `server/scripts/provisionRestaurant.js` and `server/scripts/seedDemo.js` . The new scripts follow `seedDemo.js` in driving the real API in-process.
4. The request shapes in `docs/API-CONTRACT.md` for: settings, restaurant profile, tables, stations, categories, menu items with variants and add-ons, users, payment methods and accounts.

5. `server/tests/helpers/goldenDay.js` from P10, which already sets up a restaurant through the API.
-

6. Part A. The setup script

`server/scripts/setupRestaurant.js`, run as:

```
npm run setup:restaurant -- --config
setup/caffeza.json --owner-phone 98xxxxxxx
npm run setup:restaurant -- --config
setup/caffeza.json --owner-phone 98xxxxxxx --
apply
```

Add `"setup:restaurant"` to the server and root `package.json` files, the same way `db:indexes` was added.

6a. How it runs

1. It starts the app in-process and signs in as the owner through `POST /auth/login`, the way `seedDemo.js` drives the API. It asks for the owner's password at the terminal, with the characters hidden, and never reads it from a file or a flag.
2. It reads the config file and validates the whole file with Zod before changing anything. Every problem is listed at once.
3. **Without `--apply` it changes nothing.** It prints what it would do, section by section: create, update, or already as wanted.

4. With `--apply` it does it, through the API only, never by writing documents directly.
5. It is safe to run again. Everything is matched by name: a table, station, category, payment method code, account or user that already exists is updated to match the file, or left alone if it already matches. Nothing is ever deleted or deactivated by this script.
6. Staff passwords: a new user gets a generated password, printed once at the end in a table, exactly as `provisionRestaurant.js` prints the owner's. An existing user's password is never touched.
7. At the end it prints a summary: counts created, updated, unchanged, and every `TO CONFIRM` value it skipped.

6b. What the config file holds

```
{
  "restaurant": { "name", "legalName", "gstIn",
    "fssaiLicenseNumber", "address", "contactPhone"
  },
  "settings": { "businessDayStartsAtMinutes",
    "features", "receipt", "delivery", "discounts",
    "dayClose" },
  "stations": [ { "name", "displayOrder",
    "printsTickets" } ],
  "categoryStations": { "Italian Coffees":
    "Beverages" },
  "defaultStation": "Live Kitchen",
  "tables": { "section": "Cafe", "names":
    ["Table 1", "..."], "seats": 4 },
  "paymentMethods": [ { "code", "name", "kind",
    "orderTypes", "platformCode", "tallyLedgerCode",
    "commissionBps", "displayOrder" } ],
  "accounts": [ { "name",
```

```
"openingBalanceInPaise" } ],  
  "staff": [ { "name", "phone", "role",  
    "station" } ]  
}
```

Rules for the file:

1. Any value written as the string "TO CONFIRM" is skipped, reported in the summary, and never sent to the API.
2. The invoice series is **not** in this file. It is set by hand on cutover day, as docs/G0-LIVE.md section 4 says. If settings.invoice appears in the file, refuse to run.
3. categoryStations maps category names to station names. A category in the menu that is not in the map goes to defaultStation .
4. A staff member with a "TO CONFIRM" phone is skipped, because a login needs a phone.

7. Part B. The menu import

server/scripts/importMenu.js , run as:

```
npm run import:menu -- --file setup/caffeza-  
menu.csv --owner-phone 98xxxxxxx  
npm run import:menu -- --file setup/caffeza-  
menu.csv --owner-phone 98xxxxxxx --apply
```

Same sign-in, dry run, --apply , and summary as the setup script. Share the sign-in and printing code between the two scripts rather than copying it.

7a. The CSV format

One row per item, or per size of an item that comes in sizes:

```
category,item,size,price,gst_percent,available
Italian Coffees,Caffe Latte,,220.00,5,yes
Pizza,Half & Half Pizza,,380.00,5,yes
Cold Beverages,Water Bottle,,47.61,5,yes
Pizza,Margherita,Regular,280.00,5,yes
Pizza,Margherita,Large,420.00,5,yes
```

And a second, optional file for add-ons, `--addons`
`setup/caffeza-addons.csv` :

```
item,addon,price,available
Caffe Latte,Extra Shot,40.00,yes
```

Rules:

1. Read CSV with a small, careful parser of your own or Node's built-ins, handling quoted fields with commas inside. Do not add a package for it.
2. Prices are rupees with up to two decimals, converted with `rupeesToPaise` from `money.js` . Anything else is an error for that row.
3. `gst_percent` becomes `taxRateBps` , so 5 becomes 500. Only 0, 5, 12, 18 and 28 are accepted.
4. Rows with the same category and item and a size become one item with variants. The item's own price is its first size's price.
5. Categories are created in the order they first appear, which becomes their display order. Items keep their file order within a category.

6. Every row problem is reported with its line number, and nothing is written if any row has a problem.
7. Re-running: an existing item, matched by name within its category, gets its price, GST rate, sizes and availability updated to match the file. Items on the menu but not in the file are listed as "not in this file" and left alone.

8. Part C. Caffeza's files

Create `setup/caffeza.json` from `docs/CAFFEZA-PROFILE.md` :

1. Restaurant: name "Cafezza", GSTIN `24AARFT4546K1ZM` , city Gandhinagar, state Gujarat. Legal name, FSSAI number, address and phone are `"TO CONFIRM"` .
2. Settings: business day start 300, inventory and attendance off, platform GST collected by the platform, cashier platform discounts off, the blind count on.
3. Stations: Live Kitchen, then Beverages, neither printing tickets until Caffeza confirms.
4. `categoryStations` : the Beverages list from section 8 of the profile. Default station Live Kitchen.
5. Tables: section "Cafe", "Table 1" to "Table 35" without "Table 13", seats 4 each until the floor plan is confirmed.
6. Payment methods: the eight from section 10 of the profile, with their Tally codes. Cash, Card and UPI keep the built-in codes. Zomato Gold, Dine out and Easydiner are `PLATFORM` , dine-in only. zomato and Swiggy are

PLATFORM , delivery only, linked to their platform codes.

Every commissionBps is "TO CONFIRM" .

7. Accounts: "E-210 Office" and "W-330 Office", opening balances "TO CONFIRM" .

8. Staff: the five captains as WAITER , "Counter" as CASHIER , "Live Kitchen" and "Beverages" as KITCHEN with their stations. Every phone is "TO CONFIRM" .

Create setup/caffeza-menu.csv with every item and price listed in section 9 of the profile, under the categories they appear in on the golden day in docs/TEST-DATA.md , so staging has a realistic menu for training until Caffeza sends their full menu.

Put a comment row at the top, starting with # , saying it is a partial menu and must be replaced by Caffeza's full export.

The parser skips lines starting with # .

Create setup/README.md explaining both scripts, both file formats, the dry run, and the order to run them in.

9. Tests

New file server/tests/setupScripts.test.js . Test the parsing and planning logic as plain functions, and the apply step against the in-memory database.

1. A setup file with every section applies cleanly to a fresh restaurant, and a second run reports everything unchanged.
2. Every "TO CONFIRM" value is skipped and listed. A staff member with no phone is not created.

3. A file containing `settings.invoice` is refused.
4. The dry run makes no change to the database.
5. The menu CSV in section 7a creates the right categories, items, sizes, prices in paise, and GST in basis points, with Water Bottle at 4761.
6. A quoted item name containing a comma parses correctly.
7. A row with a bad price, a GST of 7, or a missing category is reported with its line number, and nothing is written.
8. Re-importing with one price changed updates only that item.
9. `setup/caffeza.json` and `setup/caffeza-menu.csv` both pass validation in a dry run, and the counts match: 34 tables, 2 stations, 8 payment methods, 2 accounts.
10. Applying both Caffeza files to a fresh restaurant gives 34 active tables in section "Cafe", and Italian Coffees routed to Beverages.

Run the full suite at the end. Every test that passed before must still pass.

10. Non-negotiable rules that apply

"Secrets live in `.env`. Never commit a real secret." No password is ever written to a file or accepted as a flag.

"Store all money as whole paise integers." Prices go through `rupeesToPaise`.

"GST rates are settings, never hardcoded." The rate comes from the file per item.

"Check permissions on the server for every endpoint." The scripts sign in as the owner and go through the API, so every permission and validation applies.

"Never create a bill in production to test something." These scripts create no orders and no bills.

11. Checks and golden day

No money arithmetic changes.

The menu file reuses the golden day's prices, so a staging restaurant set up from it can reproduce golden day bills by hand during training.

12. Docs to update

1. `docs/DEPLOYMENT.md` section 7: replace the setup step with the exact commands, dry run first, then `--apply`.
2. `docs/GO-LIVE.md` section 4: add "Run the menu import again with Caffeza's final menu, as a dry run, and read the summary before cutover."
3. `docs/PROJECT-STATE.md` :
 1. The date line.
 2. "Current stage": "Next: P12, cloud deployment."
 3. Decision log, dated today: "Restaurants are set up from one JSON file and one menu CSV, through the real API, with a dry run by default. The scripts never delete anything, never set the invoice series, and never handle a password except the owner's, typed at the terminal. | Staging and production must be

set up identically, and cutover-day changes stay deliberate."

4. "What changed recently": a P11 entry at the top, and the oldest moved to the archive.

4. `docs/prompts/README.md` : mark P11 as Done.
-

13. Out of scope

Reading Caffeza's own menu export format. Their file arrives later, and is converted to the CSV format above. Excel files. Save as CSV first. Images, descriptions or dietary labels on menu items. Deleting or deactivating anything. Setting the invoice series.

14. Done when

1. `npm test` passes, with before and after counts recorded.
2. `npm run lint` and `npm run build` pass.
3. Locally: provision a test restaurant, run both scripts as a dry run, read the plan, run them with `--apply`, and open the order screen to see the Caffeza menu and 34 tables.
4. Every doc in section 12 is updated.
5. Commits on `main`, one line each, for example:
`add restaurant setup script`
`add menu import`

```
add caffeza setup files
```

```
update docs for p11
```

6. Push `main`.

7. Print a short summary: commits, files added, test counts before and after, and the dry-run output of both Caffeza files.